Day 33 – Circular Queue

🔵 Circular Queue (Easy Explanation)

👉 Normal queue me problem kya hoti hai?
Jab elements dequeue hote hain, front aage badh jata hai, aur peeche empty space waste ho jata hai.

👉 Solution: Circular Queue

➡️ Isme last position ke baad fir se starting position use hoti hai
➡️ Queue circular form me behave karti hai (like circle 🔄)

🧠 Real Life Example

Imagine:

5 chairs in a circle

Agar last chair fill ho gayi, next person fir first chair pe baith sakta hai

---

📌 Important Conditions

1. Queue Full

(rear + 1) % size == front

2. Queue Empty

front == -1

⚙️ Operations

➤ Enqueue (Insert)

Agar empty → front = rear = 0

Otherwise → rear = (rear + 1) % size

➤ Dequeue (Delete)

Agar ek element hai → front = rear = -1

Otherwise → front = (front + 1) % size

💻 Java Code (Simple + Interview Ready)

```java
package Day_33;

class CircularQueue {
    int front, rear, size;
    int arr[];

    CircularQueue(int size) {
        this.size = size;
        arr = new int[size];
        front = -1;
        rear = -1;
    }

    // Enqueue
    void enqueue(int x) {
        if ((rear + 1) % size == front) {
            System.out.println("Queue Overflow");
            return;
        }

        if (front == -1) { // First element
            front = rear = 0;
        } else {
            rear = (rear + 1) % size;
        }

        arr[rear] = x;
        System.out.println(x + " inserted");
    }

    // Dequeue
    void dequeue() {
        if (front == -1) {
            System.out.println("Queue Underflow");
            return;
        }

        System.out.println(arr[front] + " deleted");

        if (front == rear) { // Only one element
            front = rear = -1;
        } else {
            front = (front + 1) % size;
        }
    }

    // Display
    void display() {
        if (front == -1) {
            System.out.println("Queue is empty");
            return;
        }

        int i = front;
        while (true) {
            System.out.print(arr[i] + " ");
            if (i == rear) break;
            i = (i + 1) % size;
        }
        System.out.println();
    }

    public static void main(String[] args) {
        CircularQueue q = new CircularQueue(5);

        q.enqueue(10);
        q.enqueue(20);
```

```
        q.enqueue(30);
        q.enqueue(40);
        q.enqueue(50);

        q.display();

        q.dequeue();
        q.dequeue();

        q.enqueue(60);
        q.enqueue(70);

        q.display();
    }
}

```

🔍 Dry Run (Short)

Queue size = 5

Step 1:

Insert: 10 20 30 40 50
Queue full

Step 2:

Delete: 10 20
Space free at front

Step 3:

Insert: 60 70
(Back to starting index 🔄)

Final:

30 40 50 60 70

⚡ Key Advantage

✅ Space waste nahi hota
✅ Efficient memory usage
✅ Better than normal queue